

Atividade: Derivação de um Código a partir da Gramática de uma Linguagem de Programação

1. Linguagem escolhida:

A linguagem de programação escolhida para esta atividade foi Java.

Java é uma linguagem de programação amplamente utilizada e possui uma especificação formal mantida pela Oracle, chamada Java Language Specification (JLS). Essa documentação apresenta, entre outras informações, as regras gramaticais utilizadas para definir quais estruturas são sintaticamente válidas na linguagem.

A especificação oficial utilizada nesta atividade é a Java Language Specification – Java SE 26 Edition, disponível na documentação da Oracle. A JLS apresenta capítulos específicos sobre gramáticas livres de contexto, gramática léxica, gramática sintática e a notação utilizada para representar as produções. (Oracle Docs)

Fonte consultada:

Java Language Specification – Oracle

<https://docs.oracle.com/en/java/javase/26/docs/specs/jls/index.html>

A gramática apresentada pela especificação Java utiliza uma notação própria baseada em BNF/EBNF. Nela, os nomes das categorias representam símbolos não terminais, enquanto palavras-chave, operadores, separadores e tokens representam símbolos terminais. A documentação utiliza recursos como {x} para indicar zero ou mais ocorrências e [x] para indicar um elemento opcional. (Oracle Docs)

2. Código escolhido para a derivação

O trecho de código escolhido foi uma declaração de variável inteira com uma expressão aritmética:

```
int x = 1 + 2;
```

Esse código declara uma variável chamada x, do tipo inteiro (int), e atribui a ela o resultado da soma entre os valores 1 e 2.

O exemplo foi escolhido por ser pequeno, mas por possuir diferentes elementos da gramática Java, como:

- * declaração de variável;
- * tipo da variável;
- * identificador;
- * operador de atribuição;
- * expressão aritmética;
- * literais inteiros;

- * operador de adição;
 - * ponto e vírgula.
-

3. Produções selecionadas

Não é necessário utilizar toda a gramática da linguagem Java. Portanto, foram selecionadas apenas as regras relacionadas ao trecho escolhido.

A Java Language Specification define uma declaração de variável local da seguinte forma:

LocalVariableDeclarationStatement:

LocalVariableDeclaration ;

LocalVariableDeclaration:

{VariableModifier} LocalVariableType VariableDeclaratorList

LocalVariableType:

UnannType

var

VariableDeclaratorList:

VariableDeclarator {, VariableDeclarator}

VariableDeclarator:

VariableDeclaratorId [= VariableInitializer]

VariableDeclaratorId:

Identifier [Dims]

VariableInitializer:

Expression

ArrayInitializer

Essas regras aparecem na seção da especificação referente às declarações de variáveis locais. A regra LocalVariableDeclarationStatement, por exemplo, determina que uma declaração de variável local é formada por uma LocalVariableDeclaration seguida do terminal ;. (Oracle Docs)

Para representar a operação de soma, a gramática de expressões do Java define a produção:

AdditiveExpression:

MultiplicativeExpression

AdditiveExpression + MultiplicativeExpression

AdditiveExpression - MultiplicativeExpression

Portanto, uma expressão aditiva pode ser simplesmente uma expressão multiplicativa ou pode conter uma soma ou uma subtração. (Oracle Docs)

Para os números utilizados no exemplo, podemos chegar aos literais inteiros por meio das expressões primárias:

Primary:

- PrimaryNoNewArray
- ArrayCreationExpression

PrimaryNoNewArray:

- Literal

...

Literal:

- IntegerLiteral
- FloatingPointLiteral
- BooleanLiteral
- CharacterLiteral
- StringLiteral
- TextBlock
- NullLiteral

Dessa forma, os valores 1 e 2 são reconhecidos pela gramática como IntegerLiteral. (Oracle Docs)

4. Significado das principais regras

A regra:

LocalVariableDeclarationStatement

Representa uma declaração completa de uma variável local, incluindo o ponto e vírgula.

A regra:

LocalVariableDeclaration

define a estrutura principal da declaração, contendo o tipo da variável e o seu declarador.

A regra:

LocalVariableType

representa o tipo da variável. No código escolhido, esse tipo é int.

A regra:

VariableDeclarator

representa a variável que está sendo declarada, incluindo seu nome e, opcionalmente, o valor utilizado para inicializá-la.

A regra:

VariableInitializer

representa o valor inicial atribuído à variável. No exemplo, o inicializador é a expressão:

1 + 2

A regra:

AdditiveExpression

representa expressões que utilizam os operadores de soma + ou subtração -.

Por fim, IntegerLiteral representa um valor inteiro escrito diretamente no código, como 1 e 2.

5. Derivação do código

A derivação será iniciada a partir da estrutura necessária para representar uma declaração de variável local.

O símbolo inicial adotado para o trecho escolhido é:

<LocalVariableDeclarationStatement>

Aplicando a primeira produção:

<LocalVariableDeclarationStatement>

⇒ <LocalVariableDeclaration> ;

Agora aplicamos a produção de declaração de variável local:

⇒ <LocalVariableType> <VariableDeclaratorList> ;

Como será declarada apenas uma variável:

⇒ <LocalVariableType> <VariableDeclarator> ;

O tipo da variável escolhida é int:

⇒ int <VariableDeclarator> ;

Em seguida, utilizamos a produção de VariableDeclarator com um identificador e um inicializador:

⇒ int <VariableDeclaratorId> = <VariableInitializer> ;

O identificador utilizado será x:

⇒ int x = <VariableInitializer> ;

O inicializador é uma expressão:

⇒ int x = <Expression> ;

Para gerar a soma desejada, chegamos a uma expressão aditiva:

⇒ int x = <AdditiveExpression> ;

Aplicamos então a regra:

AdditiveExpression

→ AdditiveExpression + MultiplicativeExpression

Assim:

⇒ int x = <AdditiveExpression> + <MultiplicativeExpression> ;

O primeiro operando será o literal inteiro 1:

⇒ int x = 1 + <MultiplicativeExpression> ;

O segundo operando será o literal inteiro 2:

⇒ int x = 1 + 2 ;

Portanto, chegamos à sequência final:

int x = 1 + 2;

6. Derivação resumida

A derivação também pode ser representada de maneira mais compacta:

<LocalVariableDeclarationStatement>

⇒ <LocalVariableDeclaration> ;

⇒ <LocalVariableType> <VariableDeclaratorList> ;

⇒ int <VariableDeclarator> ;

⇒ int <VariableDeclaratorId> = <VariableInitializer> ;

⇒ int x = <Expression> ;

⇒ int x = <AdditiveExpression> ;

⇒ int x = <AdditiveExpression> + <MultiplicativeExpression> ;

⇒ int x = 1 + <MultiplicativeExpression> ;

⇒ int x = 1 + 2 ;

Resultado:

int x = 1 + 2;

7. Símbolos terminais

Os símbolos terminais são aqueles que aparecem efetivamente no código produzido e não precisam ser substituídos por outras regras gramaticais.

No exemplo utilizado, podem ser considerados terminais:

```
int
x
=
1
+
2
;
```

O int representa o tipo inteiro.

O x é o identificador da variável.

O = é o operador de atribuição.

Os símbolos 1 e 2 são literais inteiros.

O símbolo + representa o operador de soma.

O símbolo ; determina o final da declaração.

8. Símbolos não terminais

Os símbolos não terminais representam categorias sintáticas da linguagem. Durante o processo de derivação, eles são substituídos pelas produções definidas na gramática.

Entre os não terminais utilizados estão:

```
LocalVariableDeclarationStatement
LocalVariableDeclaration
LocalVariableType
VariableDeclaratorList
```

VariableDeclarator
VariableDeclaratorId
VariableInitializer
Expression
AdditiveExpression
MultiplicativeExpression
IntegerLiteral

Por exemplo, VariableInitializer não aparece diretamente no código Java. Ele representa uma categoria da gramática que pode ser substituída por uma Expression, que depois é derivada até chegar à expressão concreta `1 + 2`.

9. Resultado final

O código obtido ao final da derivação foi:

```
int x = 1 + 2;
```

A derivação demonstra que um código escrito em uma linguagem de programação não é apenas uma sequência aleatória de caracteres. Sua estrutura precisa obedecer a um conjunto de regras formais estabelecidas pela gramática da linguagem.

Primeiramente, foi utilizada a regra responsável pela declaração de uma variável local. A partir dela, foram definidas as estruturas correspondentes ao tipo da variável, ao seu identificador e ao seu valor inicial.

Depois, o valor inicial foi derivado como uma expressão. A regra de expressões aditivas permitiu inserir o operador `+`, enquanto os operandos foram derivados até chegar aos literais inteiros `1` e `2`.

Assim, sucessivas aplicações das regras da gramática transformaram os símbolos não terminais em símbolos terminais até produzir a sequência de tokens correspondente ao código Java válido.

10. Conclusão

A realização desta atividade permitiu relacionar os conceitos estudados em gramáticas formais com uma linguagem de programação utilizada na prática.

Foi possível observar que elementos como declarações, identificadores, operadores, tipos e expressões são definidos através de regras de produção. Durante uma derivação, parte-se de um símbolo não terminal e são realizadas substituições sucessivas até que sejam obtidos apenas símbolos terminais.

No exemplo analisado, as regras da gramática da linguagem Java permitiram gerar a declaração:

```
int x = 1 + 2;
```

Apesar de ser um código simples, ele demonstra como uma gramática formal determina a estrutura sintática aceita por uma linguagem de programação.

Referências

ORACLE. The Java Language Specification – Java SE 26 Edition. Disponível em: <https://docs.oracle.com/en/java/javase/26/docs/specs/jls/index.html>. Acesso em: 26 ago. 2026.

ORACLE. Java Language Specification – Chapter 14: Blocks, Statements, and Patterns. Disponível em: <https://docs.oracle.com/javase/specs/jls/se26/html/jls-14.html>. Acesso em: 26 ago. 2026.

ORACLE. Java Language Specification – Chapter 15: Expressions. Disponível em: <https://docs.oracle.com/en/java/javase/26/docs/specs/jls/jls-15.html>. Acesso em: 26 ago. 2026.